Week 1: GitHub Copilot and Javascript Fundamentals

This document covers the effective use of **GitHub Copilot within VS Code** while building and enhancing a **Social Media Posts (ProConnect)** JavaScript application. It is designed to help you build practical fluency with Copilot's core capabilities — including inline suggestions, Chat (Ask and Agent modes), managing models and context. Along the way, you will also develop a foundational understanding of how Large Language Models (LLMs) work, their limitations, and how to prompt them effectively.

By the end of this session, you will be able to:

- Use GitHub Copilot's inline code suggestions effectively
- Navigate Copilot Chat (Chat Panel and Inline Chat)
- Apply Ask Mode and Agent Mode appropriately
- Perform multi-file edits using structured prompts
- Manage chat sessions and context windows strategically
- Understand model limitations and validate AI-generated code
- Understand simple Git workflows to commit your work

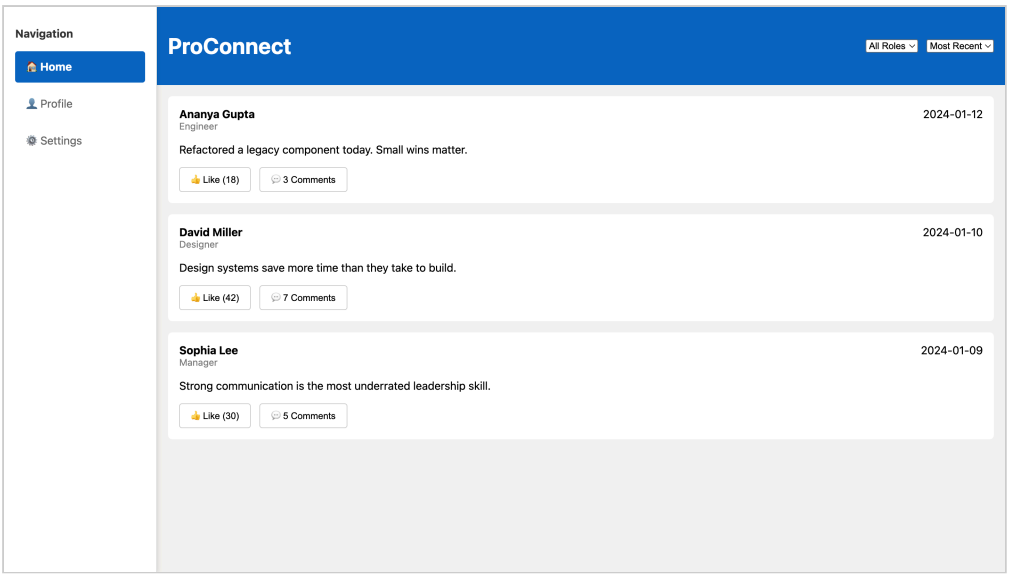
Prerequisites:

- GitHub account with a Copilot subscription (accept invitation)
- VS Code, Node.js and Git installed
- GitHub Copilot Chat VS Code extension installed and Signed in
- JavaScript/HTML/CSS knowledge from the Prework

1. Project Description and Setup

ProConnect is a LinkedIn-style application that displays a list of professional posts on a webpage. Each post includes the following details: Author Name and Role, Post Content, Like button with number of Likes, Comments and Timestamp of post creation.

The application uses JavaScript to manage how the posts appear on the page and for user interaction. It allows users to view posts, like them, filter posts by role, and sort them by date. The UI is built with HTML and styled with CSS to create a clean and modern look.



Project Setup and Structure

1. Download and extract the assignment starter code from the zip file provided on Olympus.
2. Initialize a new Git repository (either by running `git init` in the terminal or by using the VS Code Git extension). (See the Git in VSCode tutorial)
3. As you make changes throughout this tutorial, remember to regularly commit your work.

The project has the following files:

1. index.html – Defines the structure of the webpage and displays the social posts UI
2. styles.css – Contains all styling rules for layout, colors, and visual appearance
3. index.js – Manages post data, renders posts dynamically, and handles user interactions and application logic
4. misc.js - A scratchpad file for testing Copilot features

Running the App: This is a plain HTML/JS project — there is no build step. To view the app in a browser, install the **Live Server** extension in VS Code, then right-click `index.html` and choose **"Open with Live Server"**. Alternatively, you can drag `index.html` directly into any browser window.

2. Inline Code Suggestions in GitHub Copilot

Github Copilot's Inline Suggestions, also known as "autocomplete" or "ghost text," provides real-time code suggestions as you type. The suggestions appear as grayed-out text that appears as you type. This is the most immediate way Copilot assists you.

How it works:

1. You start typing code
2. Copilot analyzes your *context* (current file, comments, etc)
3. Suggestions appear as faded/gray text
4. Press `Tab` to accept, or keep typing to ignore

Open the file `misc.js` and try out these exercises:

Exercise 1: Comment-Driven Development

Type the following comment, press `Enter` and wait - Copilot should suggest the entire function.

```
// Function to calculate the factorial of a number
```

Expected behavior: Copilot should suggest the complete function body. Note how Copilot infers the appropriate programming language from the file extension.

Press `Tab` to accept the suggestion. You may need to press `Tab` multiple times to accept the full suggestion.

Exercise 2: Array Operations Type this and pause after the typing the comment:

```
const numbers = [1, 2, 3, 4, 5];

// Filter even numbers
```

Expected behavior: Copilot suggests the filter function to extract the even numbers.

Press `Tab` to accept the suggestion. You may need to press `Tab` multiple times to accept the full suggestion.

Exercise 3: Code from function signature Type only a function signature and pause:

```
function reverseString(str)
```

Press `Enter` and wait - Copilot should suggest the entire function.

Inline Suggestions Keyboard Shortcuts

Action	Windows/Linux	macOS
Accept suggestion	`Tab`	`Tab`
Accept next word	`Ctrl+→`	`Cmd+→`
Dismiss suggestion	`Esc`	`Esc`

Tips for Better Inline Suggestions

1. Write descriptive comments first

```
// Validate email format using regex, return true if valid
function validateEmail(email) {
```

2. Use meaningful variable and function names

```
// Good - Copilot understands intent
function calculateMonthlyMortgagePayment(principal, rate, years) {

// Bad - unclear intent
function calc(p, r, y) {
```

Behind the Scenes: How GitHub Copilot Works

To use Github Copilot effectively, it's important to understand **what it is doing under the hood**. In this section, you will understand some fundamental concepts around Generative AI and Large Language Models that will help you use Copilot more effectively and avoid common pitfalls.

Github Copilot is powered by Large Language Models (LLMs). These models do not "think" like humans — they predict likely next pieces of text (called *tokens*) based on patterns learned during an earlier training phase.

At its core:

- An LLM predicts the **next token** in a sequence.
- It was trained on large amounts of public text and code.
- It does not execute your code.
- It does not truly "understand" meaning.
- It generates statistically likely continuations.

For example, if you type the following prompt

```
Write a poem about a happy cat
```

in ChatGPT, it generates a poem because the underlying LLM has seen many poems and sentences about happy cats during training. It is not "creating" in the human sense — it is generating text that is statistically likely to follow from the prompt based on its training data.

Similarly when you type:

```
function calculateTotal(price, taxRate) {
```

the LLM powering Copilot predicts what usually comes next in similar code contexts. Think of it as an advanced autocomplete engine powered by probability.

Tokens

LLMs process text in a sequence of characters known as **tokens**.

- A token can be a word, a set of characters or a combination of words and punctuation.
- In English text, 1 token ≈ 3–4 characters (roughly). So 100 tokens is roughly 75 words.
- Code often uses more tokens due to symbols and formatting.

Exercise: Go to the website https://platform.openai.com/tokenizer and try pasting some text / code snippets to see how they are tokenized. For e.g.

```
function factorial(n) {
  if (n <= 1) return 1;
  return n * factorial(n - 1);
}
```

We will come back to these concepts later in the tutorial. Let us continue our exploration of Github Copilot features.

Next Edit Suggestions

In the file `misc.js`, go back to the following code snippet from Exercise 2:

```
const numbers = [1, 2, 3, 4, 5];

// Filter even numbers
const evenNumbers = numbers.filter((num) => num % 2 === 0);
console.log(numbers, evenNumbers);
```

Change `numbers` to `numList` in the first line and see how Copilot suggests the next edits to fix the code. Press Tab to accept the suggested changes.

This is an example of how Copilot can chain multiple edits together based on your initial change.

Note: The Inline Suggestions and the Next Edit features can be enabled or disabled in the Copilot settings. Click on the Copilot icon in the bottom-left corner of VS Code to enable/disable these features based on your preference.

3. Github Copilot Chat

The Chat Panel in Github Copilot provides a conversational interface to interact with AI. It is a dedicated sidebar for longer conversations. There is also an Inline Chat mode for quick interactions directly in the editor which we will later explore.

Chat Panel

Click the 'Toggle Chat' icon in the Title Bar of VS Code. Alternatively, use `Ctrl+Alt+I` (Windows/Linux) or `Ctrl+Cmd+I` (macOS).

Chat Modes

Ask, Plan and Agent Mode are different ways to interact with Copilot Chat. Each mode is designed for specific types of interactions, from simple questions to complex multi-step tasks.

Mode	Primary Use Case	Copilot's Behavior	Best Used When
Ask Mode	Ask questions to understand concepts, explore and understand code, errors, or best practices	Provides explanations, answers, examples, and clarifications	You want understanding or guidance without modifying code
Agent Mode	Make specific, localized changes to selected code or files	Applies targeted edits that you review and approve	You know exactly what needs to change and want fast implementation
Plan Mode	Design solutions, explore approaches, and break down problems before coding	Generates structured plans, architectures, and step-by-step strategies	You want to think before coding or align on an approach

In this session, we will focus on **Ask Mode** and **Agent Mode**. The other modes will be explored in future sessions.

3.1 Ask Mode

The Ask Mode in Copilot Chat allows you to ask questions and get explanations, examples, and guidance from the AI. It is ideal for learning, understanding code, debugging, and exploring new concepts.

Exercise 1 : Ask Mode

1. Open the Chat Panel (`Ctrl+Alt+I` / `Ctrl+Cmd+I`). Ensure you are in **Ask Mode**.
2. Type this prompt:

```
I am an experienced Java developer, but new to Javascript. Explain
let, const, and var in Javascript in a way that connects it to Java
concepts.

 Follow this structure: high-level intuition, analogy to Java,
similarities, differences, side-by-side code examples and when to use
it.
```

3. Read the response and ask a follow-up:

```
What are some common mistakes that Java developers make?
```

4. Notice how Copilot maintains the context of the conversation.

Chat Panel Features

- **Conversation History**: Chat remembers previous messages in the conversation. Each new message builds on prior context.
- **Code Blocks**: Responses include syntax-highlighted code
- **Buttons to Copy**: Easily copy code snippets into your clipboard or directly into your editor

Exercise 2 : Ask Mode

1. Start a new chat session (`Ctrl+N` / `Cmd+N`). Starting a new session clears prior context. It is **strongly recommended** to start fresh chat sessions for new topics to keep the context clear and not let old context interfere with questions on new topics.

2. Type this prompt:

```
Give me an overview of this codebase
```

3. Review the response to understand the project structure and functionality of the ProConnect application.

4. Explore follow-up questions like:

```
How does the rendering work?
```

or select a section of code in 'styles.css' and ask:

```
Explain the CSS rules in the selected code
```

Inline Chat

Inline Chat lets you ask quick questions and make edits directly in the editor without leaving your code. You can open an Inline Chat session by placing your cursor in the code and pressing `Ctrl+I` (Windows/Linux) or `Cmd+I` (macOS).

Exercise 1 : Inline Chat

Open the file `misc.js` and copy the following function into it:

```
function processUserData(users) {
  let result = [];
  for (let i = 0; i < users.length; i++) {
    if (users[i].age >= 18) {
      result.push({
        name: users[i].name,
        email: users[i].email,
      });
    }
  }
  return result;
}
```

A : Explain Code

1. Select the entire function
2. Press `Ctrl+I` / `Cmd+I`
3. Type: `Explain what this function does`
4. Review the explanation provided by Copilot.

B : Refactor Code

1. Select the entire function
2. Press `Ctrl+I` / `Cmd+I`
3. Type: `Refactor this using modern JavaScript syntax`
4. When Copilot suggests changes, you will see a diff view of the proposed edits.
5. Review the suggested code changes carefully and accept them if they look fine.

C : Add Error Handling

1. Select the entire function

2. Press `Ctrl+I` / `Cmd+I`
3. Type: `Add input validation and error handling`
4. Review the suggested code changes carefully and accept them if they look fine.

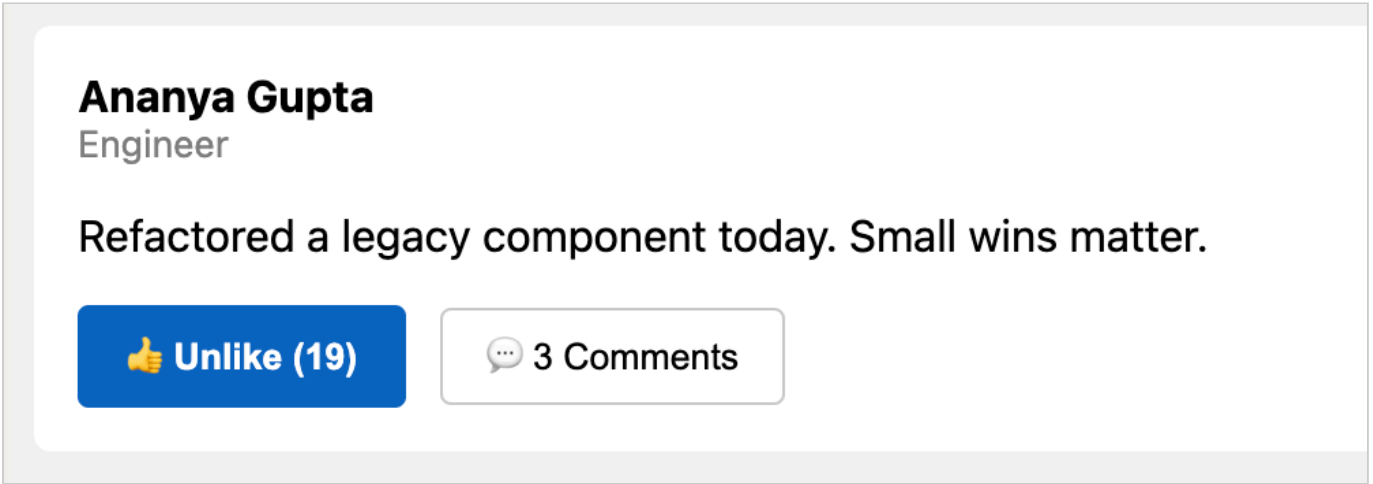
Exercise 2 : Inline Chat

In our ProConnect app, let us add a new feature to Toggle Likes using Inline Chat. When a user clicks the Like button, it should toggle between Like and Unlike, and update the like count accordingly.

1. Open `index.js` and find the code that handles the Like button click event.
2. Place your cursor inside index.js at the Like button click handler where the toggle logic should be added.
3. Press `Ctrl+I` / `Cmd+I` to open Inline Chat.
4. Type the following prompt:

   ```
   Add functionality to toggle between Like and Unlike when the Like button is clicked, and update the like count accordingly
   ```

5. Review the suggested code changes carefully and accept them if they look fine.
6. Expected Outcome: When you click the Like button, it should toggle between Like and Unlike, and the like count should increase or decrease accordingly. The UI should update to reflect the current state.



Commit Changes

Commit the changes you made to the `misc.js` and `index.js` files using Git. You can use the Git extension in VS Code. Write a meaningful commit message like "Add error handling to processUserData function" and "Implement Like button toggle functionality".

Chat Panel vs Inline Chat: When to Use Each

Use the Inline Chat for quick questions and small edits directly in the code editor. It is great for understanding specific code snippets, refactoring functions, or adding small features. For anything more complex, use the Chat Panel.

Context Window

Let us now explore some important concepts around how Copilot uses context to generate responses.

Every model has a **maximum token limit** (context window) that it can process at once. The context window includes:

- System message
- Your prompt (User message)
- Any Selected files
- Previous chat history
- Copilot's response

(We will look at the system and user messages in more detail in future weeks.)

The context ensures that Copilot has the necessary information to generate relevant responses. However, if the total tokens exceed the model's limit, older parts of the conversation or context may be truncated.

Large files or long conversations that bloat the context window may degrade quality of responses. It is important to manage context effectively.

That's why starting a **New Chat (`Ctrl+N` / `Cmd+N`)** often improves results.

Fundamentals of Prompt Engineering

A "prompt" is a request message that you send to Github copilot. Prompt engineering refers to a few principles that help craft effective prompts that can significantly increase your chances of getting better quality responses. Here are some practical principles to keep in mind when crafting prompts for Copilot:

1. Be specific: When prompting the LLM be as specific as possible about what you want. Vague prompts lead to vague answers. For example:

- Bad: "Improve this code."
- Better: "Refactor this code. Extract helper functions and reduce nested conditionals. Use latest ES6 syntax where relevant"

2. Provide the *right* context: It is important to provide relevant context to guide the model. However, avoid providing too much context that can dilute the important information.:

- Add the relevant file(s) or select the relevant code.
- Avoid adding unrelated large files that dilute the context.

3. Constrain the solution: Adding constraints can help guide the model to produce more relevant and usable responses. For example:

- Specify libraries/frameworks (or “use React functional components.”).
- Specify style constraints (ES6+, no external dependencies).

4. Assign a role or persona to Copilot: Sometimes it helps to give a specific role to Copilot to guide the style and approach of the response. For example:

- “You are a senior software architect. ”
- “You are a React expert. ”

5. Use Examples: It can be helpful to provide examples of the desired output format or style. For example:

- “Write a function that formats an input date. For example, input: ‘2024-06-01’, output: ‘June 1, 2024’.”

5. Iterate and Refine: Prompting is an iterative process. Do not spend inordinate amount of time trying to build a perfect prompt in the first attempt. If the first response isn’t what you wanted, refine your prompt and try again. You can also ask follow-up questions to clarify or improve the response.

3.2 Agent Mode

The Agent Mode in Copilot Chat is a powerful feature that allows you to make multiple changes across one or more files based on a single prompt. It is ideal for implementing specific features, refactoring code, or making coordinated changes across files. Most of the time, you will want to use Agent Mode for implementing features or making changes that require edits in multiple places.

Exercise: Agent Mode Let us add a Sidebar to our ProConnect app with navigation links to Home, Profile, and Settings. We will use Agent Mode to implement this feature.

1. In the Chat Panel, switch to **Agent Mode** using the dropdown at the bottom of the panel.
2. Add the following files as context:
    - `index.html`
    - `styles.css`
    - `index.js`

    You can add files by clicking on the "Add Context" button in the chat and selecting the files. Alternatively, you can type in # followed by the file name in the prompt to add them as context.
3. Type in the following prompt:

```
Add a Sidebar to the app with navigation links to Home, Profile, and
Settings. Fix the Sidebar on the left side of the screen and have
```

```
styled navigation links.
```

4. Review the suggested code changes carefully. You can see a diff of the proposed changes before accepting them.

Note how we provided specific files as context for the Agent Mode. This helps Copilot focus on the relevant code and produce more accurate suggestions. This is especially important for larger codebases where unrelated files could add noise to the context.

4. Model Limitations and Managing Usage

Hallucinations

A significant limitation of LLMs is their tendency to **hallucinate** — confidently generating incorrect or nonsensical information. This is because they predict likely tokens, not factual correctness. As a result, Github Copilot can :

- Invent functions that don't exist
- Use outdated or incorrect syntax
- Generate incorrect test cases
- Confidently produce wrong code

Trust but Verify

As users of AI generated code, it is **CRITICAL** to always carefully review and validate any code produced by Copilot. Never assume it is correct. Run the code and test it to ensure it works as intended.

Determinism and Variability

Another significant feature of LLMs is that they are inherently probabilistic. As a result:

- The same input prompt will likely produce different outputs on different attempts.
- Further, different models will respond differently to the same input.

This is expected behavior. If the first answer isn't great, refine the prompt and regenerate the response.

Security and Responsible Use

It is important to use Github Copilot responsibly and securely. Since Copilot generates code based on patterns learned from public data, there are potential risks around data privacy and intellectual property. Be mindful to:

- Understand your organization's AI usage policy.
- Never paste secrets, API keys, or credentials into prompts.
- Avoid sharing proprietary code in public model contexts.

Choosing AI Models, Managing Usage and AI Credits

GitHub Copilot supports multiple AI models, each with different strengths. The list of available models is continuously evolving, with new models being added and older ones being deprecated over time. The models differ in their capabilities, speed, and cost (in terms of AI credit consumption).

Note: Model availability depends on your subscription plan. Free tier has limited options.

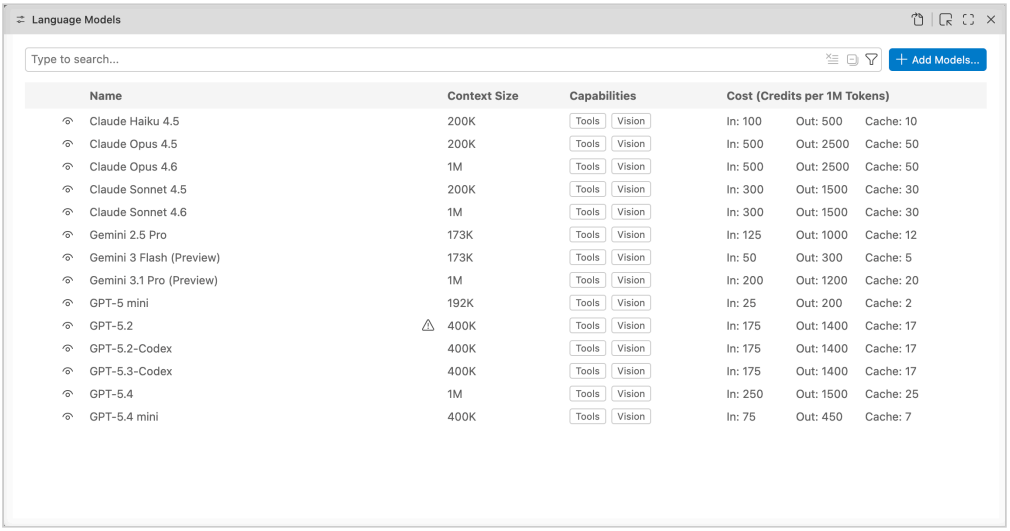
How to Select and Change Models

At the bottom of the Chat Panel, you can select which model to use for generating responses. You can switch models at any time during a conversation.

Model performance can vary based on the specific prompt and context. Experimenting with different models for the same task can help determine which one produces the best results for your particular use case.

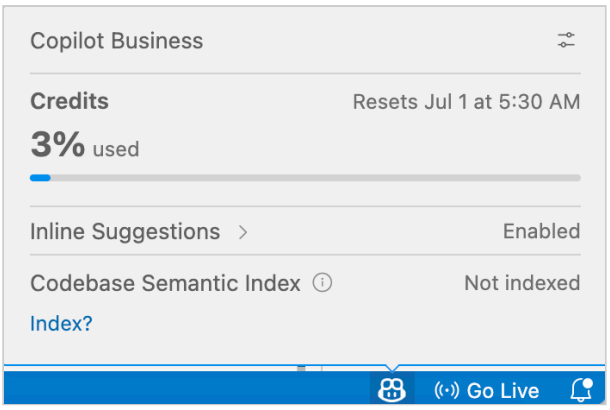
IMPORTANT: Understanding AI Credits

GitHub Copilot's models consume **AI Credits** when invoked. 1 AI credit = $0.01 USD. Here is the current list of models and their costs:



Note the premium models consume a large number of credits (for e.g. Claude Opus 4.6 consumes 500 and 2500 AI credits for 1M input and output tokens respectively). This means that using premium models for long conversations or large code generations can quickly consume your monthly allotted credits.

You have a Copilot Business subscription which gives you around 2000 AI credits per month. You can check your AI credit usage by clicking on the Copilot icon in the bottom-right corner of VS Code. The Credits section will show you how many credits you have used and what percentage of your monthly limit you have left. Note that there might be some delay in updating the credit usage in the UI.



Tips to manage AI credits effectively:

1. Use the cheap models (GPT-5-mini preferably for most tasks, else Gemini 3 Flash (preview), GPT-5.4-mini or Claude Haiku).
2. Do NOT use the premium models (Claude Opus 4.5+, GPT-5.2+) for now.
3. Monitor your usage regularly to avoid running out of premium requests.
4. Strictly limit your weekly usage to less than 25% of your monthly limit to ensure you have enough credit for the assignments and projects throughout the month. (So maximum usage of 25% in week1, 50% in week2, 75% in week3 and so on).

🔗 Additional Resources

- GitHub Copilot Documentation
- VS Code Copilot Features
- Copilot Chat Cheat Sheet
- AI Model Comparison